

software Component Design Project

Project Title: Mini Shop Application

Software Component Design

Date: [17-5-2026]

Application Description:

Our project is a small console-based Mini Shop application written in Java. The application allows the user to log in using a username and password, view a list

of products, choose products with their pricings to add to the cart, select a payment method, and place an order. The application is simple, and it is designed using object-oriented programming and design patterns.

The user of the application is a normal customer who wants to buy products from a small shop. The user can choose products such as a laptop, mouse, keyboard, and headphones. After choosing the products, the application calculates the total price and lets the user pay using either cash or credit card. Then the order goes through different states such as pending, paid, and shipped. The application also sends notifications using both email and SMS.

Main Features of the Application:

The application includes the following features:

1. User login:

The user enters a username and password. The application checks the username first and then checks the password.

2. Product list:

The user can see a list of available products with prices.

3. Add products to cart:

The user can choose products by entering the product number. The selected products are added to the cart.

4. Payment:

The user can choose between cash payment and credit card payment.

5. Order workflow:

The order moves through different states: Pending, Paid, and Shipped.

6. Notifications:

The application sends order messages using email and SMS.

7. Logging:

The application uses a logger to print important application events.

Design Pattern Explanations

Factory Pattern

Why:

We used Factory to create the correct payment method without creating it directly in **Main**.

Where:

Used in **PaymentFactory**, **PaymentStrategy**, **CashPayment**, and **CreditCardPayment**.

What if:

We could use **if** statements in **Main**, but Factory keeps object creation cleaner and easier to extend.

Singleton Pattern

Why:

We used Singleton because the app only needs one shared logger.

Where:

Used in **Logger**, and called in **Main** using **Logger.getInstance()**.

What if:

We could create many logger objects, but one shared logger is simpler and avoids repeated objects.

Bridge Pattern

Why:

We used Bridge to separate notification type from the sending method.

Where:

Used in **Notification**, **OrderNotification**, **MessageSender**, **EmailSender**, and **SMSSender**.

What if:

We could create separate classes for every notification type, but Bridge avoids too many repeated classes.

Decorator Pattern

Why:

We used Decorator to add a discount to a product without changing the original **Product** class.

Where:

Used in **ProductDecorator** and **DiscountDecorator**.

What if:

We could add discount directly inside **Product**, but Decorator keeps extra features separate and optional.

Composite Pattern

Why:

We used Composite to represent a category that contains products.

Where:

Used in **CategoryComponent**, **CategoryComposite**, and **ProductLeaf**.

What if:

We could store products in a normal list, but Composite gives a clearer structure for categories and products.

Command Pattern

Why:

We used Command to turn “add to cart” into a separate action object.

Where:

Used in **Command** and **AddToCartCommand**.

What if:

We could call **cart.addProduct()** directly, but Command separates the action from the user input code.

Strategy Pattern

Why:

We used Strategy because the user can choose different payment methods.

Where:

Used in **PaymentStrategy**, **CashPayment**, and **CreditCardPayment**.

What if:

We could put all payment logic in one method, but Strategy keeps each payment method in its own class.

State Pattern

Why:

We used State because the order changes from Pending to Paid to Shipped.

Where:

Used in **OrderState**, **PendingState**, **PaidState**, **ShippedState**, and **Order**.

What if:

We could use a simple String for order status, but State keeps each status behavior organized.

Observer Pattern

Why:

We used Observer so email and SMS notifications react automatically when the order changes.

Where:

Used in **Observer**, **EmailNotification**, **SMSNotification**, and **Order**.

What if:

We could call each notification manually, but Observer makes adding new notifications easier.

Chain of Responsibility Pattern

Why:

We used Chain of Responsibility to check username first, then password.

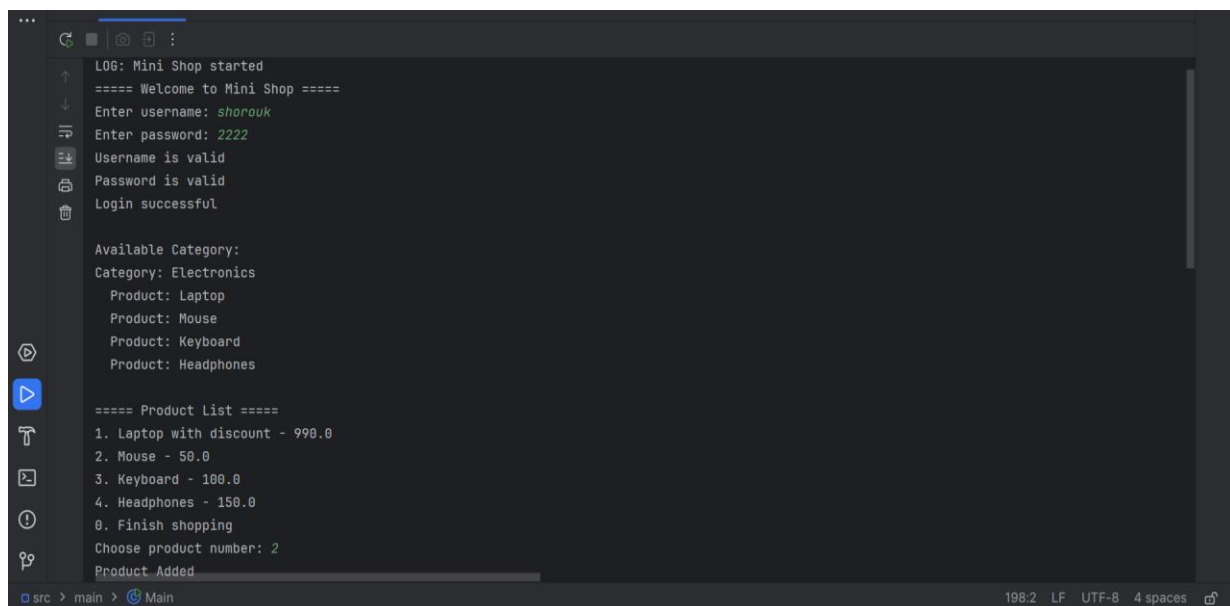
Where:

Used in **Handler**, **UsernameHandler**, and **PasswordHandler**.

What if:

We could write all login checks in **Main**, but the chain keeps each check separate and easier to modify.

Program Output:



```
LOG: Mini Shop started
==== Welcome to Mini Shop ====
Enter username: shorouk
Enter password: 2222
Username is valid
Password is valid
Login successful

Available Category:
Category: Electronics
Product: Laptop
Product: Mouse
Product: Keyboard
Product: Headphones

==== Product List ====
1. Laptop with discount - 990.0
2. Mouse - 50.0
3. Keyboard - 100.0
4. Headphones - 150.0
0. Finish shopping
Choose product number: 2
Product Added
```

src > main > Main 198:2 LF UTF-8 4 spaces

```

Mouse added to cart.

===== Product List =====
1. Laptop with discount - 990.0
2. Mouse - 50.0
3. Keyboard - 100.0
4. Headphones - 150.0
0. Finish shopping
Choose product number: 0

Cart items:
- Mouse : 50.0
Total = 50.0

===== Payment Method =====
1. Cash
2. Credit Card
Choose payment method: 1
Paid using Cash: 50.0

Email: Order state changed to: Pending
SMS: Order state changed to: Pending
Changing order from Pending to Paid

src > main > Main 198:2 LF UTF-8 4 spaces
===== Payment Method =====
1. Cash
2. Credit Card
Choose payment method: 1
Paid using Cash: 50.0

Email: Order state changed to: Pending
SMS: Order state changed to: Pending
Changing order from Pending to Paid
Email: Order state changed to: Paid
SMS: Order state changed to: Paid
Changing order from Paid to Shipped
Email: Order state changed to: Shipped
SMS: Order state changed to: Shipped

Email Sender: Thank you, shorouk. Your order was received.
SMS Sender: Your order is now being processed.
LOG: Mini Shop finished

Process finished with exit code 0

src > main > Main 198:2 LF UTF-8 4 spaces

```

UML Diagram:

